

Document:	P1232
Date:	October 8, 2018
Audience:	SG1
Authors:	Lee Howes < lw@fb.com > Eric Niebler < eniebler@fb.com > Kirk Shoop < kirkshoop@fb.com > Lewis Baker < lbaker@fb.com > Robert Geva < ryg@fb.com >

Integrating executors with the standard library through customization.

Summary

To fully benefit from executors we need to integrate them into the standard library. As a potential user of the parallel and concurrent algorithms in the standard Facebook is excited to see the standard library be able to serve part of the purpose that custom accelerated algorithm libraries currently solve.

Unfortunately, we do not believe that the bulk execution as defined in [P0443](#) alone is adequate to provide performance optimal to each runtime or platform backing the executor passed to the algorithm. Indeed, there is already discussion in the direction of adding nested parallel to this, minimally exposing the work-group or thread-group concept from the OpenCL and CUDA execution models. Even that added nesting will not be adequate: we will need to expand the set of algorithms and queries available on executors over time to be able to expect that the standard algorithms provide a continued high level of performance using these facilities.

To avoid this, and because we already have a set of algorithms defined in the standard, we propose taking a direct approach and customizing the standard library directly and hence to a large degree allowing the executor author to directly customize algorithms they are able to customize, to achieve peak performance on hardware and runtimes executor authors are aware of rather than only those standard library authors are able to map to.

Background

[P0443](#) defines bulk execution on executors. During the Bellevue ad-hoc meeting we agreed that the long-term direction proposed in [P1194](#) to move to a Sender/Receiver model will be beneficial, but that in the short term we should focus on one way execution only, with a goal to support the standard library and upcoming networking API as defined in the networking TS. Unfortunately, bulk execution on executors as defined relies on the standard algorithms implementing a significant amount of customization work on executors to hope to achieve competitive performance. The definition of and amount of shared state, the shape of the parallel execution, and the chunk size computed by each worker are up to the algorithm.

It is therefore hard to see how a standard library can create a portable implementation of the parallel algorithms across the range of possible executors. Furthermore, optimally we would like to see passing an executor provided by some accelerator hardware or optimized library vendor to allow that library to take the role of implementing the optimized algorithm, because the authors of that library are in a position to fully understand the tradeoffs of implementing such an algorithm far better than a universal library author can hope to. Yet the current definition expects both parties to be involved in that optimization, leading to a potentially unscalable need for a standard library vendor to understand all possible executors that are passed in to provide a suitable implementation of the algorithm. In particular, without extending the standard library code directly it is not possible to optimize the structure of the entire standard algorithm for a given executor. The opportunity to optimize should be offered to the author of an executor, such that importing a new executor really provides the application author with the opportunity to benefit from any knowledge that executor author has that would improve the behavior of the standard algorithms.

[P1019](#) proposes the addition of `.on(Executor)` to the execution policies, to add a `.executor()` operation to the policies to return the default and a few other minor changes. These all appear entirely reasonable. We would, however, drop the `bulk_execution_requirement` proposed, because this is an unnecessary complication when generalizing the algorithms. The policy covers the assumptions the user has made of the parameters to the algorithm, should be obeyed by the executor, and is available to the customization, but beyond that adds little.

The proposal

To avoid an $n \times m$ relationship between standard library and executor authors, and to allow future scaling of algorithm performance as new executor vendors arise, without the need for us to reimplement the standard algorithms or, worse, to ask the standard library vendor to reimplement them for us, we should allow the algorithms to be directly customized on the executor.

By customizing the algorithms directly new executor implementors can improve the performance of algorithms by providing their own customizations, independent of updates to the standard library. We should apply the same approach to range-based parallel algorithms in future, consistent with the current work on ranges as in [P0896](#).

While there is scope for an $m \times n$ expansion here, in practice a standard library vendor can specify in documentation what dependence graph they are using to implement their algorithms. In the worst case, this amounts to implementing all independently, but it might, for example, mean implementing them all in terms of `for_each` which would allow an executor vendor to target `for_each` first and all algorithms can benefit. This one-way information flow from standard library implementor to executor implementor will be much easier to scale than expecting standard library implementors to also optimize for the range of available executors.

`std::async` should also have executor support. As a concrete future concept is not likely to arrive in C++20, we propose that even with executors `std::async` should be an eager, `std::future`-returning operation and should not customize its future type. Any equivalent of `async` in a world of new futures with support for laziness should be a new algorithm.

`std::async` could be defined in terms of directly enqueueing to the executor, but this would require that we enqueue a task that satisfies a `std::promise`. It is conceivable that an implementation might be capable of returning something convertible to `std::future` or via other means, so the means of producing the `std::future` is implementation-defined under customization. Similarly the parallel algorithms are synchronous, but we would prefer to avoid relying on a blocking enqueue property to achieve that because we wish to be able to disallow executors that block on enqueue entirely from our codebase, given how dangerous they can be if passed deep into asynchronous code. To that end we would prefer to customize the algorithms to use their own blocking mechanism internally with the aim of being able to customize our code without the risk of introducing dangerous blocking executors.

Changes to `future.syn`

Add to `<future>` header synopsis:

```
template <class Executor, class F, class... Args>
    future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(Executor ex, F&& f, Args&&... args);
```

Changes to futures.async

Add:

```
template <class Executor, class F, class... Args>
    future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
    async(Executor ex, F&& f, Args&&... args);
```

To *effects* add: The third function calls `std::execution::async_e(std::move(ex), std::forward<F>(f), std::forward<Args>(args)...)`

Add a new section *Customization Points*.

The name `async` denotes a *customization point object*. The expression `execution::async_e(E, F, Args...)` for some expressions `E`, `F` and the list `Args...` is expression-equivalent to the following:

- `static_cast<decltype(async_e(E, F, Args...))>(async_e(E, F, Args...))` if that expression is well-formed when evaluated in a context that does not include `execution::async_e` but does include the lookup set produced by argument-dependent lookup (6.4.2).
- Otherwise if `is_executor_v<E>` is true then: `execution::async_e(E, F, Args...)`.
- Otherwise `execution::async_e(E, F, Args...)` is ill-formed.

Changes to 23.19.2 Header synopsis

Execution policies should be typed on the executor.

Replace:

```
namespace std::execution {
// 23.19.4, sequenced execution policy
class sequenced_policy;
// 23.19.5, parallel execution policy
class parallel_policy;
// 23.19.6, parallel and unsequenced execution policy
class parallel_unsequenced_policy;

// 23.19.7, execution policy objects
inline constexpr sequenced_policy seq{ unspecified };
inline constexpr parallel_policy par{ unspecified };
inline constexpr parallel_unsequenced_policy par_unseq{ unspecified };
```

With:

```
namespace std::execution {
// 23.19.4, sequenced execution policy
template<class Executor>
class sequenced_policy;
// 23.19.5, parallel execution policy
template<class Executor>
class parallel_policy;
// 23.19.6, parallel and unsequenced execution policy
template<class Executor>
class parallel_unsequenced_policy;
```

Changes to execpol.seq

We add the `executor()` and `on(Executor)` operations.

Replace:

```
class execution::sequenced_policy { unspecified };
```

With:

```
template<class Executor>
class execution::sequenced_policy
{
public:
    Executor executor() const;

    template<class OtherExecutor>
        sequenced_policy<OtherExecutor>
        on(const OtherExecutor& ex) const;
};
```

Introduce a member function `sequenced_policy::executor()` :

```
Executor executor() const;
```

Returns: A copy ex of this execution policy's associated executor.

Introduce a member function `sequenced_policy::on()` :

```
template<class OtherExecutor>
sequenced_policy<OtherExecutor> on(const OtherExecutor& ex) const;
```

Returns: An execution policy object policy identical to `*this` with the exception that the returned execution policy's associated executor is a copy of `ex` . `execution::is_execution_policy<decltype(policy)>::value` is true.

Changes to `execpol.par`

We add the `executor()` and `on(Executor)` operations.

Replace:

```
class execution::parallel_policy { unspecified };
```

With:

```
template<class Executor>
class execution::parallel_policy
{
public:
    Executor executor() const;

    template<class OtherExecutor>
        parallel_policy<OtherExecutor>
        on(const OtherExecutor& ex) const;
};
```

Introduce a member function `parallel_policy::executor()` :

```
Executor executor() const;
```

Returns: A copy ex of this execution policy's associated executor.

Introduce a member function `parallel_policy::on()` :

```
template<class OtherExecutor>
parallel_policy<OtherExecutor> on(const OtherExecutor& ex) const;
```

Returns: An execution policy object policy identical to `*this` with the exception that the returned execution policy's associated executor is a copy of `ex`. `execution::is_execution_policy<decltype(policy)>::value` is true.

Changes to `execpol.parunseq`

We add the `executor()` and `on(Executor)` operations.

Replace:

```
class execution::parallel_unsequenced_policy { unspecified };
```

With:

```
template<class Executor>
class execution::parallel_unsequenced_policy
{
public:
    Executor executor() const;

    template<class OtherExecutor>
    parallel_unsequenced_policy<OtherExecutor>
    on(const OtherExecutor& ex) const;
};
```

Introduce a member function `parallel_unsequenced_policy::executor()` :

```
Executor executor() const;
```

Returns: A copy `ex` of this execution policy's associated executor.

Introduce a member function `parallel_unsequenced_policy::on()` :

```
template<class OtherExecutor>
unspecified on(const OtherExecutor& ex) const;
```

Returns: An execution policy object policy identical to `*this` with the exception that the returned execution policy's associated executor is a copy of `ex`. `execution::is_execution_policy<decltype(policy)>::value` is true.

Changes to `execpol.objects`

Replace:

```
inline constexpr execution::sequenced_policy seq{ unspecified };
inline constexpr execution::parallel_policy par{ unspecified };
inline constexpr execution::parallel_unsequenced_policy par_unseq{ unspecified };
```

With:

```
inline constexpr execution::sequenced_policy<unspecified> seq{ unspecified };
inline constexpr execution::parallel_policy<unspecified> par{ unspecified };
```

```
inline constexpr execution::parallel_unsequenced_policy<unspecified> par_unseq{ unspecified };
```

Changes to algorithms.parallel.overloads

Add: Parallel algorithms overloads taking an `ExecutionPolicy` not defined by the implementation will dispatch to a customization point object in the `std::execution` namespace.

Using `all_of` as an example, this modification should be propagated to all other algorithms with `ExecutionPolicy` overloads.

Changes to alg.all_of

Add: The execution policy overload will dispatch to `execution::all_of_e(std::forward<ExecutionPolicy>(exec), first, last)` for any execution policy not defined by the implementation.

Add a new section *Customization Points*.

The name `all_of_e` denotes a *customization point object*. The expression `execution::all_of_e(EP, F, L)` for some expressions `EP`, `F` and `L` is expression-equivalent to the following:

- `static_cast<decltype(all_of_e(EP, F, L))>(all_of_e(EP, F, L))`, if that expression is well-formed when evaluated in a context that does not include `execution::all_of_e` but does include the lookup set produced by argument-dependent lookup (6.4.2).
- Otherwise if `is_execution_policy_v<decay_t<EP>>` is true then `execution::all_of_e(EP, F, L)`
- Otherwise `execution::all_of_e(EP, F, L)` is ill-formed.

Repeat the above definition of `all_of` for the other `ExecutionPolicy`-taking algorithms.

Open questions

- The wording for customization points will need clarifying to decide precisely the right strategy for having an overload of a pre-existing `std::` function call a customization point object without risk of conflicts. I have added `_e` to the names in the `execution` namespace as a handover from a pre-existing `std` namespace function into an `execution` customization point.
- In the text above we templated the execution policies. Due to the pre-existing ones we may want to create new policies, or make the return type of `policy.on(executor)` entirely implementation defined with the expectation that implementations use traits correctly. If so this is a minor change and we have left it out of the proposal above. Quite how we package the result of `policy.on(executor)` is fairly flexible without changing the fundamentals.